

Autoencoders

Sistemas de Inteligencia Artificial

21 de junio de 2026

ITBA — 2026

Autoencoder basico

El problema y el dataset font.h

Dataset: 32 caracteres de una fuente bitmap.

- Cada letra: grilla $7 \times 5 = 35$ **pixeles** binarios $\{0, 1\}$.
- $X \in \{0, 1\}^{32 \times 35}$ (sin normalizar: ya es binario).
- Chico y discreto $\Rightarrow$ ideal para forzar memorizacion por un cuello.

Pregunta gancho: ¿se pueden comprimir 35 pixeles a **2 numeros** y reconstruirlos con ≤ 1 **pixel** de error?

Objetivo

Aprender los **32 patrones** de 5×7 en un latente de **2 dimensiones** con un error maximo de **1 pixel incorrecto**.

Ejemplo (letra 'a'):

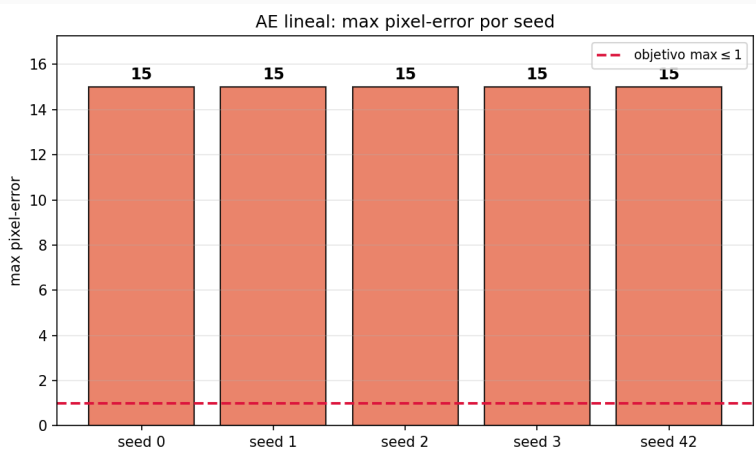
```
.....  
....#  
.####  
#...#  
#...#  
.##.#  
.....
```

Primer intento — ¿alcanza con un AE lineal?: parametros

Parametro	Valor
Datos	<code>font.h</code> (32×35)
Arquitectura	[35, 60, 40, 20, 2, 20, 40, 60, 35]
Activaciones	todas identidad (lineal)
Perdida	MSE
Optimizador	Adam, $lr = 5 \cdot 10^{-3}$
Adam $\beta_1, \beta_2, \epsilon$	0,9, 0,999, 10^{-8}
Batch	full-batch (32)
Init	Xavier
Epocas	20000
Seeds	[0, 1, 2, 3, 42]
Metrica	max pixel-error + rec-MSE

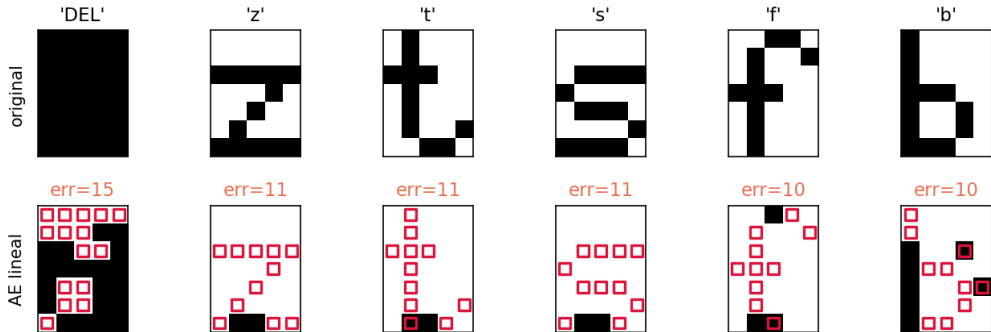
Pregunta: ¿alcanza un autoencoder *lineal* (la version mas simple) para reconstruir las 32 letras con ≤ 1 pixel de error?

Primer intento — ¿alcanza con un AE lineal?: resultados

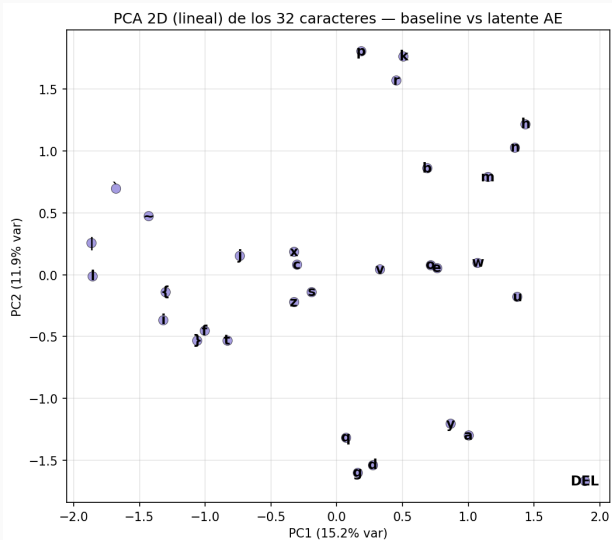


Una letra concreta: 'DEL' pierde 15 de 35 pixeles

AE lineal: peores reconstrucciones (rojo = pixel erroneo)



¿Por que falla? Converte a PCA



- Un AE *lineal* converge a PCA.
- Su latente 2D retiene solo **~27 %** de la varianza $\Rightarrow$ no separa las 32 letras.

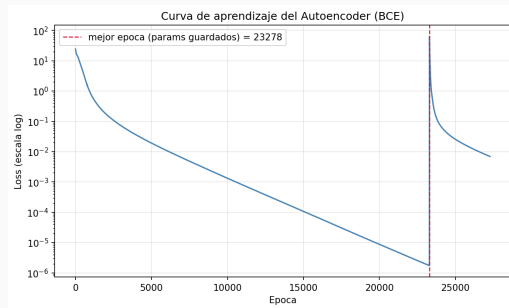
La respuesta: no linealidad — arquitectura base del AE

Misma idea simetrica con **cuello 2D**, pero **ocultas no lineales**:

35 → 60 → 40 → 20 → 2 → 20 → 40 → 60 → 35

- Ocultas **tanh**; latente **lineal** (no satura); salida **logistica** (pixeles en $(0, 1)$).
- Perdida **BCE**, **Adam**, full-batch, EarlyStopping.
- **Sin regularizacion**: queremos *sobreajustar* (memorizar) las 32 letras.

Esta es la *configuracion base* de referencia: los experimentos varian **una** pieza a la vez.



Curva de loss (BCE) de la corrida principal.

Estudio de optimizacion

Una variable a la vez, controlando el resto.

Estudio one-at-a-time: metodologia (comun a los 3 ejes)

Parametro	Valor
Datos	font.h (32×35)
Config base	[35, 60, 40, 20, 2, ...], tanh, latente lineal, BCE, Adam 10^{-3}
Adam $\beta_1, \beta_2, \epsilon$	0,9, 0,999, 10^{-8}
Batch / Init	full-batch (32) / Xavier
Epocas	6000
Seeds	[0, 1, 2, 3, 42]
Metrica	max pixel-error (media $\pm$ std)
Criterio conv	1a epoca con $\max \leq 1$

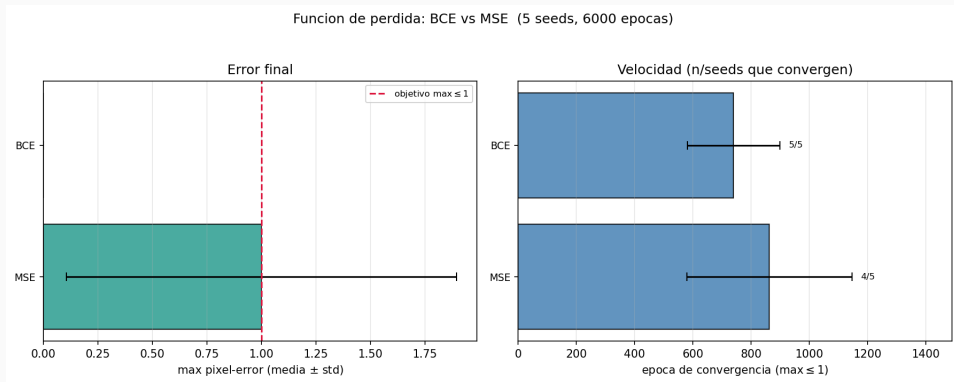
Pregunta: ¿que ingredientes permiten cruzar ≤ 1 pixel, y de forma *robusta* entre seeds?

Variamos **una** pieza a la vez:

1. perdida (BCE / MSE)
2. activacion oculta (tanh / relu / logistica)
3. arquitectura (profundidad / ancho)

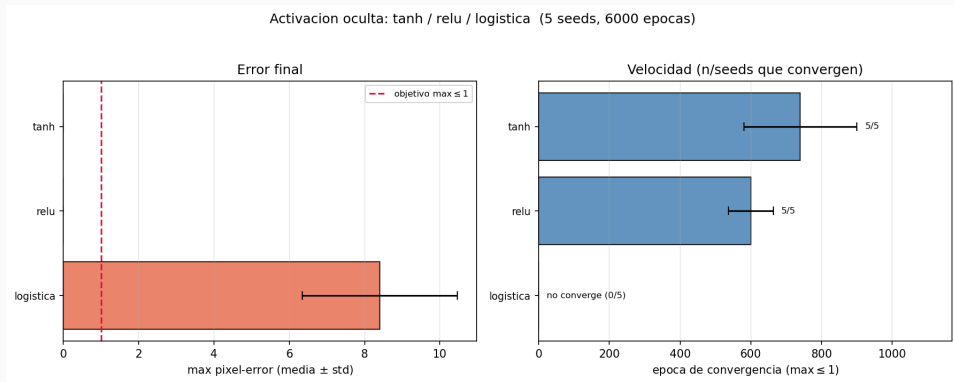
(optimizador $\times$ lr: experimento aparte.)

Eje 1 — Funcion de perdida: BCE vs MSE



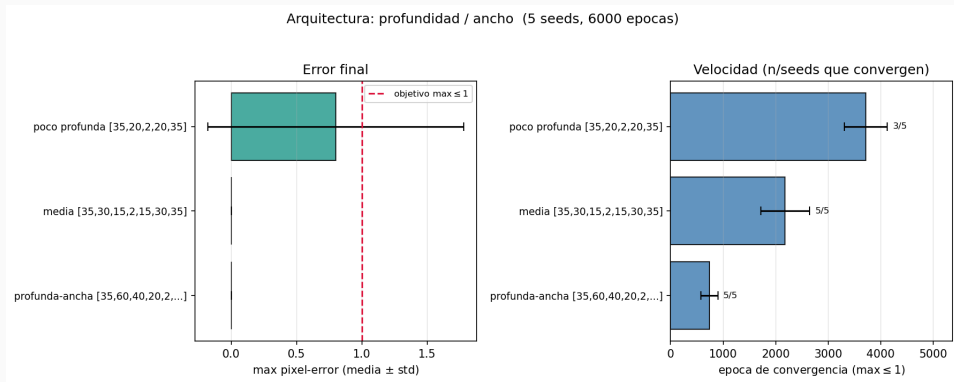
BCE 5/5 vs MSE *borderline* (4/5). Fijamos BCE.

Eje 2 — Activacion oculta: tanh / relu / logistica



tanh/relu 5/5; logistica 0/5 (satura el gradiente). Fijamos tanh.

Eje 3 — Arquitectura: profundidad / ancho



poco profunda inestable (3/5); mas profundidad/ancho $\Rightarrow$ confiable. **Fijamos la profunda-ancha** (5/5).

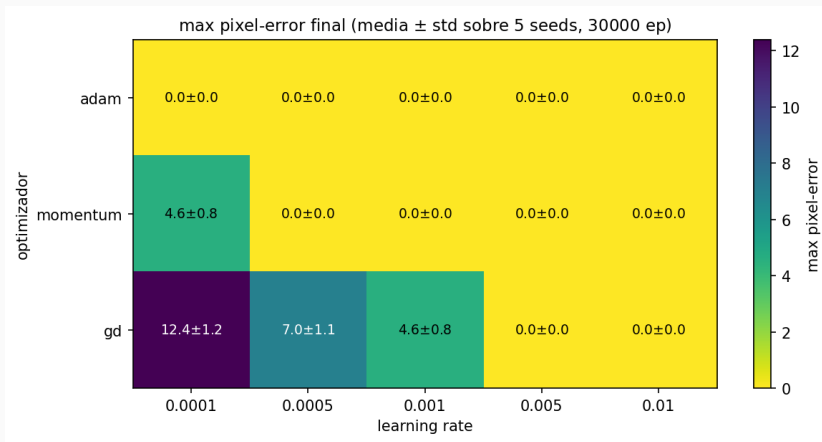
Eje 4 — Optimizador $\times$ learning rate: parametros

Parametro	Valor
Datos	font.h (32×35)
Config base	profunda-ancha, tanh, BCE
Batch / Init	full-batch (32) / Xavier
<i>Variable</i>	optimizador $\times$ lr
optimizador	GD / Momentum(0.9) / Adam
Adam $\beta_1, \beta_2, \epsilon$	0,9, 0,999, 10^{-8}
lr	10^{-4} a 10^{-2} (5 valores)
Protocolo	5 seeds $\times$ 30000 ep

Pregunta:

- ¿el lr optimo *depende* del optimizador?, ¿es Adam robusto al lr?

Eje 4 — Optimizador $\times$ learning rate (5 seeds $\times$ 30000 ep)



Adam: max= 0 en *todo* el rango de lr (robusto). GD y Momentum solo con lr altos. **Fijamos Adam.**

Resultados

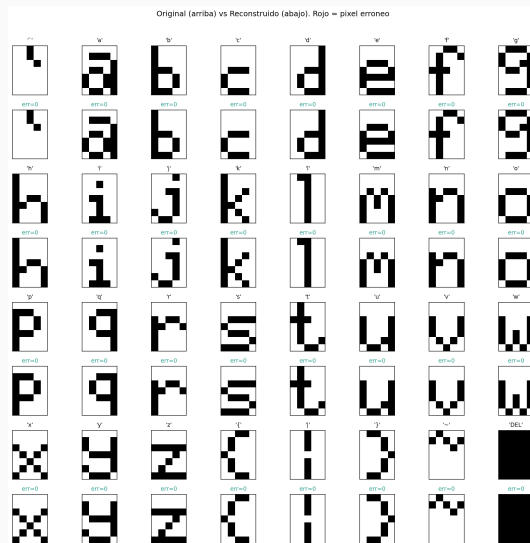
Objetivo ≤ 1 pixel: CUMPLIDO

Configuración del modelo final

Arquitectura	[35, 60, 40, 20, 2, 20, 40, 60, 35]
Activaciones	tanh / latente lineal / logistica
Perdida / Opt	BCE / Adam ($lr = 10^{-3}$)
Adam $\beta_1, \beta_2, \epsilon$	0,9, 0,999, 10^{-8}
Init / Epocas	Xavier / 30000
Early stopping	patience 4000, $\min\Delta = 10^{-7}$
Batch / Seed	full-batch (32) / 0

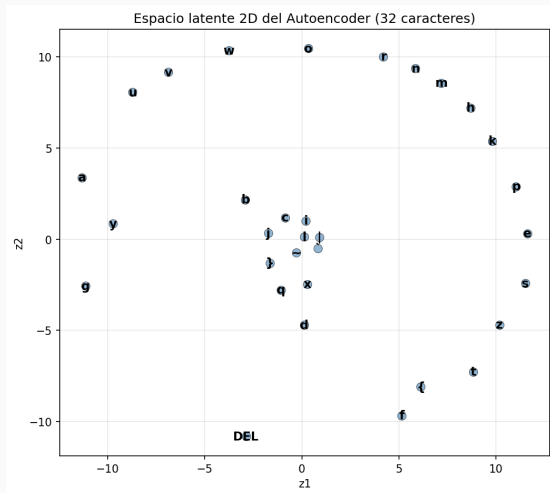
Metrica	Valor
max pixel-error	0
letras exactas	32/32
BCE final (mejor ep)	$1,74 \times 10^{-6}$

Cumplido con margen ($\max = 0$). Las 5 seeds llegan a $\max = 0$; model.npz reproduce el resultado.



Original (arriba) vs reconstruccion (abajo); rojo = pixel

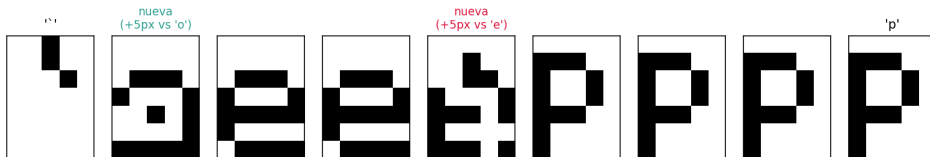
Espacio latente 2D del AE



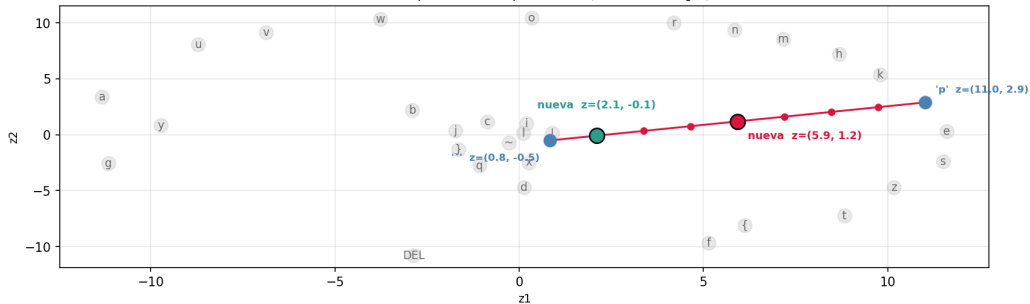
- El AE *no lineal* ubica las 32 letras en posiciones bien separadas del plano 2D.
- Mismo cuello 2D que el PCA, pero aca reconstruye **perfecto** (vs 27 % de varianza del PCA lineal).

Generar una letra nueva (interpolacion)

Generacion de letra nueva: interpolacion 'i' -> 'p' | punto medio a 5px de la letra mas cercana ('e') -> no pertenece al set

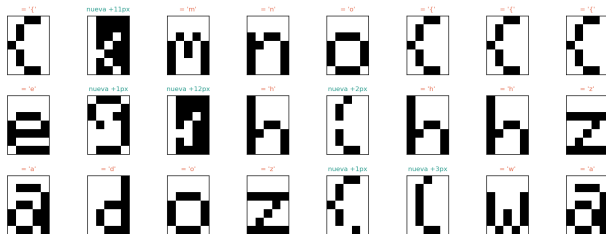


Recta de interpolacion en el espacio latente (las 32 letras en gris)

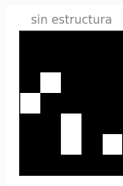


El latente del AE no es generativo

AE: 500 z random uniformes en el rango del latente -> decoder
colapsan a una letra existente: 75% | sin estructura: 1% | nuevas plausibles: 24% (el latente del AE no es generativo)



- Muestreamos 500 puntos *z* *al azar* en el rango del latente y decodificamos.
- **75 %** colapsa a una letra existente; **24 %** son "nuevas plausibles".
- El **1 %** restante es un patron *sin estructura*, que no podria considerarse una nueva letra:



Denoising Autoencoder

Parametro	Valor
Datos	font.h (32 × 35)
Tarea	entrada ruidosa → target limpio
Augmentation	online (ruido nuevo cada epoca)
Arquitectura	[35, 60, 40, 20, 8, 20, 40, 60, 35]
Activaciones	tanh (oculta), logistica (salida) cuello <i>lineal</i> (identidad)
Loss / Opt	BCE / Adam 10^{-3}
Epocas / Batch / Init	12000 / full-batch / Xavier
Ruido train	salt&pepper $p = 0,10$
Seeds	[0, 1, 2, 3, 42]

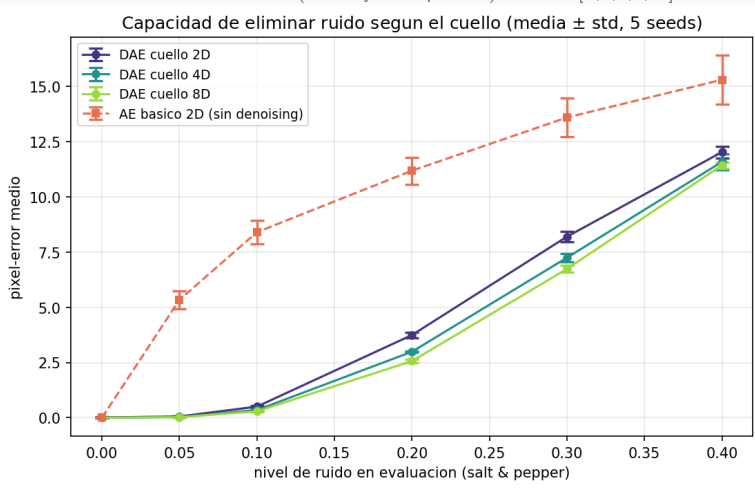
Modelos de ruido (barrido de evaluacion):

- **salt&pepper**: invierte cada pixel con prob. p .
- **gaussian**: $x + \mathcal{N}(0, \sigma^2)$, clip a $[0, 1]$.
- **masking**: pone a 0 una fraccion de pixeles.

Se entrena solo con salt&pepper; gaussian y masking miden *generalizacion* a ruido no visto.

Estudio 1 — ¿Que cuello? 2D vs 4D vs 8D

Varia: tamaño del cuello (resto fijo: ver planteo). 5 seeds [0,1,2,3,42].



2D queda corto (31.8/32 limpio); **4D satura** y **8D** es mejor por poco. Barras = std de 5 seeds.

Estudio 2 — nivel de ruido en train: parametros

Parametro	Valor
Datos	font.h (32×35)
Config base	DAE cue llo 8D (Estudio 1), lineal, tanh, BCE, Adam 10^{-3}
Entrenamiento	salt&pepper online, target limpio
Batch / Init	full-batch (32) / Xavier
Epocas	12000
Seeds	[0, 1, 2, 3, 42]
Metrica	pixel-error medio (32 letras $\times$ 20 realizaciones)

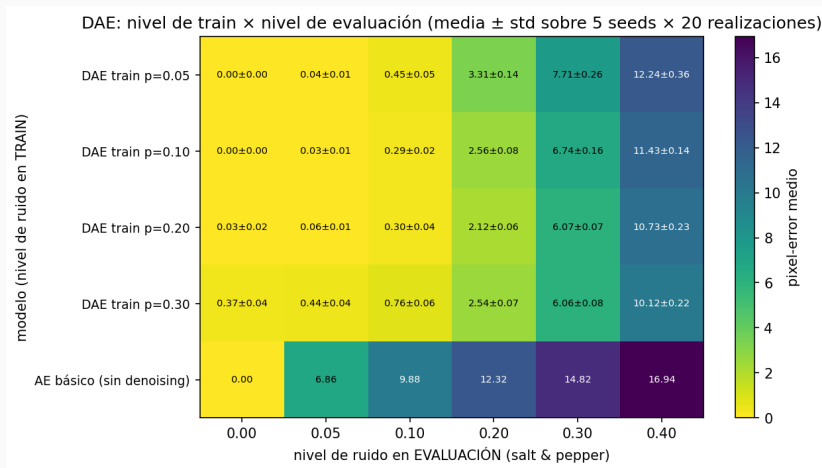
Variable: **nivel de ruido en train**

$p \in \{0,05, 0,10, 0,20, 0,30\}$ (un DAE por nivel).

Pregunta: ¿cuanto ruido conviene en train para generalizar a distintos niveles?

Eval: barriendo el ruido $0 \rightarrow 0,40$; **media** $\pm$ **std** sobre las 5 seeds.

Estudio 2 — ¿Cuanto ruido conviene en train?



Sweet spot $p \approx 0,10-0,20$: $p = 0,05$ generaliza mal a ruido fuerte; $p = 0,30$ ya degrada el limpio.

Estudio 3 — activacion del cuello: parametros

Parametro	Valor
Datos	font.h (32 × 35)
Config base	DAE cuello 8D (Estudios 1–2), tanh oculta, BCE, Adam 10^{-3}
Entrenamiento	salt&pepper online $p=0,10$, target limpio
Batch / Init	full-batch (32) / Xavier
Epocas	12000
Seeds	[0, 1, 2, 3, 42]
Metrica	pixel-error medio (32 letras × 20 realizaciones)

Variable: **activacion del cuello**

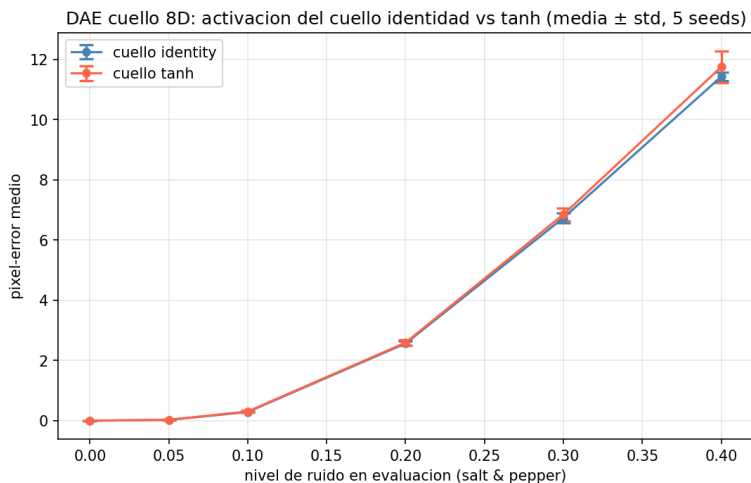
$\in \{\text{identidad}, \tanh\}$.

Pregunta: ¿la activacion del latente cambia el denoising? (en 1a el cuello era lineal).

Eval: barriendo el ruido $0 \rightarrow 0,40$; **media** $\pm$ **std** sobre las 5 seeds.

Estudio 3 — ¿Importa la activacion del cuello?

Varia: activacion del cuello, identidad vs tanh (resto fijo: ver parametros). 5 seeds.

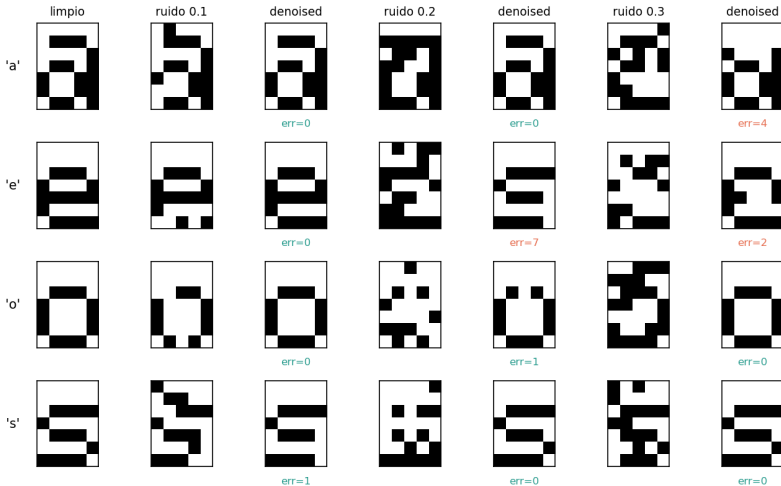


Empate: ambas reconstruyen el limpio (32/32) y dan el mismo denoising dentro del std. Identidad es igual o

Capacidad de denoising (cualitativo)

DAE final: cuello 8D, entrenado con salt&pepper $p = 0,10$.

Denoising AE — ruido 'salt_pepper': limpio / ruidoso / reconstruido



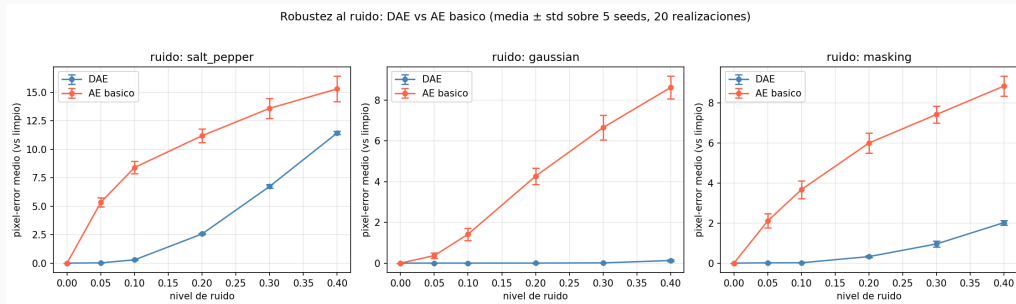
Parametro	Valor
Datos	font.h (32×35)
DAE	cuello 8D lineal, train salt&pepper $p = 0,10$ (online)
AE basico	cuello 2D lineal, <i>sin</i> ruido (input=target)
Loss / Opt	BCE / Adam 10^{-3}
Epocas	DAE 12000 / basico 30000
Seeds	[0, 1, 2, 3, 42]
Metrica	pixel-error medio (32 letras $\times$ 20 realizaciones)

Comparacion: 3 tipos de ruido

(salt&pepper, gaussian, masking),
barriendo el nivel $0 \rightarrow 0,40$.

Pregunta: ¿el denoising es un efecto
aprendido?

Robustez al ruido: DAE vs AE basico



El DAE queda **muy por debajo** en error en los 3 ruidos, incluso en *gaussian* y *masking* que **no vio en train**.

Conclusiones

- Un AE no lineal con cuello **2D** memoriza las 32 letras con **0 píxeles de error**, muy por encima del baseline PCA 2D.
- **Receta robusta**: BCE + tanh + Adam funciona bien y es insensible al lr.
- El **DAE** limpia salt&pepper hasta $p \approx 0,2-0,3$ y generaliza a ruidos no vistos en train (gaussian, masking).
- Cuello **8D lineal** es óptimo para denoising; el nivel de ruido de train tiene un sweet spot en $p \approx 0,10-0,20$.
- **Límite del AE**: el espacio latente **no tiene estructura**; muestrear puntos arbitrarios no genera letras válidas. $\Rightarrow$ necesitamos un espacio latente organizado.